

Master's Degree in Data Science
Academic Year: 2025-2026

Daniela Lorriz Bautista - id. 900074
Matteo Fumagalli - id. 894468

Foundations of Deep Learning Project

CLASSIFICATION OF AQUATIC ANIMALS

Problem

Challenge

Human sketch recognition is a challenging task in computer vision:

- Drawings are stylized, minimal, and often incomplete.
- Individuals have different drawing styles.

Dataset

The **Google Quick, Draw!** dataset is the world's largest collection of sketches, containing over 50 million drawings across 345 categories.

Objective

Train and evaluate multiple Convolutional Neural Network (**CNN**) **architectures** to classify drawings into their categories.

Focus

10 classes of aquatic animals:

crab, crocodile, dolphin, fish, frog, lobster, octopus, sea turtle, shark, and whale.

Data Extraction

1. Raw format: data is extracted from .ndjson files containing vector stroke coordinates.

2. Recognition filter: sketches not recognized by the official Google Quick, Draw! model are excluded to filter out ambiguous drawings.

3. Stroke length analysis: drawings below the 5th percentile and above the 95th percentile are discarded to reduce noise.

4. Balanced dataset: 20,000 sketches are randomly sampled from each class.

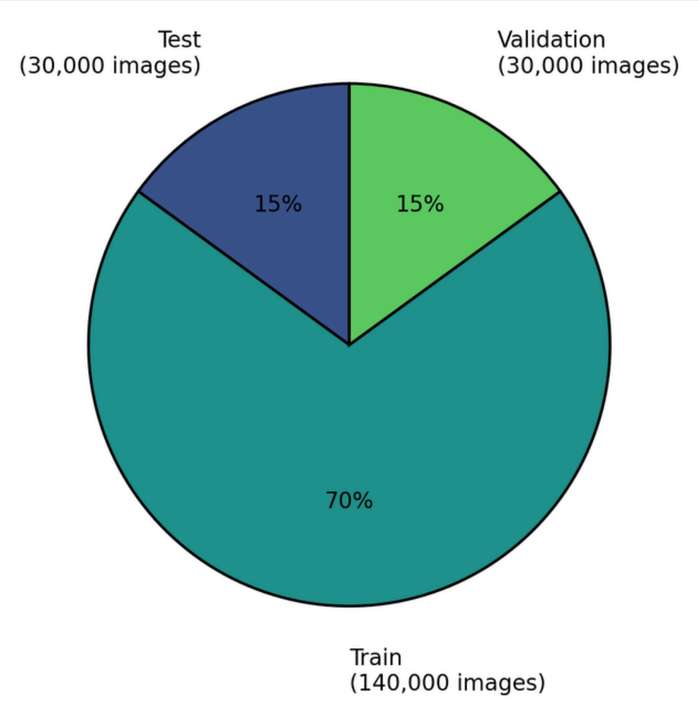
5. Coordinates extraction and scaling: vector (x, y) coordinates are extracted and rescaled to a standard 256x256 canvas as drawings vary in size and resolution depending on user's screen device.

6. Image reconstruction: drawings are centered within the canvas. Then, white lines are drawn on a black background to convert the vector paths into a grayscale image.

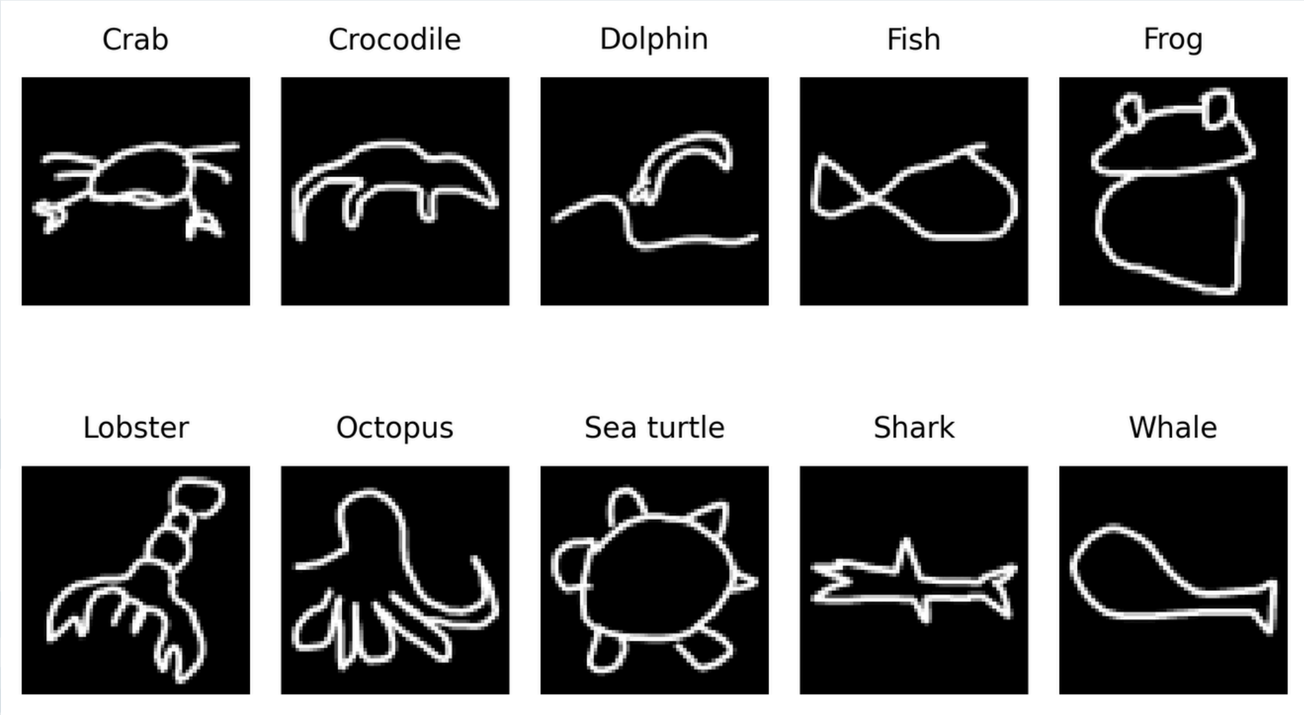
7. Image resizing: images are downsampled to 64x64 pixels.

Data Exploration

DISTRIBUTION OF IMAGES



RECONSTRUCTED IMAGES



Solution to the Problem

For the Aquatic Animal Classification task, we developed and evaluated five **CNN architectures**.

OUR STRATEGY

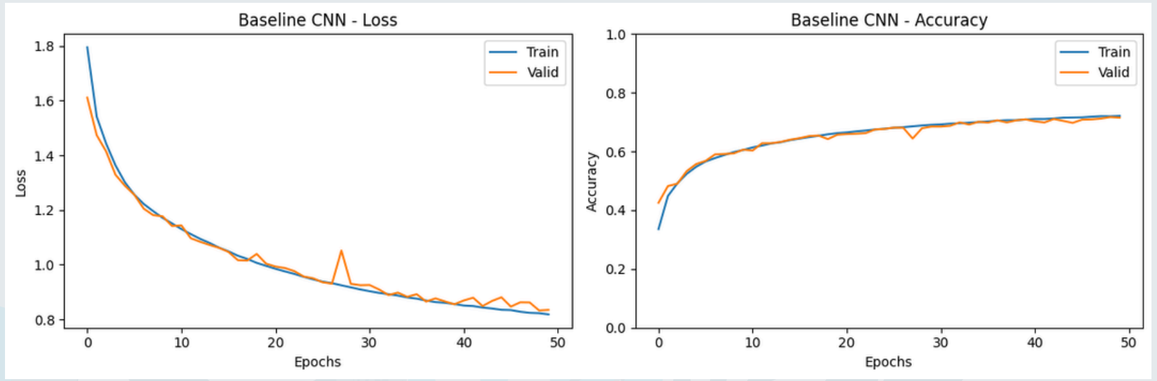
In particular, since Quick, Draw! sketches are sparse, minimal, and inconsistent across users, we adopted an **incremental complexity strategy**. We started from a lightweight baseline and progressively introduced regularization, augmentation, automated tuning, and transfer learning to find the right model capacity without overfitting.

1. **Baseline CNN:** lightweight 2-block architecture used as a reference model.
2. **Regularized CNN:** BatchNorm and Dropout added to a 3-block convolutional architecture.
3. **Augmented CNN:** Regularized architecture enhanced with data augmentation during training.
4. **Tuned CNN:** Architecture optimized through hyperparameter search using Keras Tuner.
5. **Transfer Learning:** MobileNetV2 with frozen backbone, followed by selective fine-tuning.

From Baseline to Regularized CNN

Baseline CNN

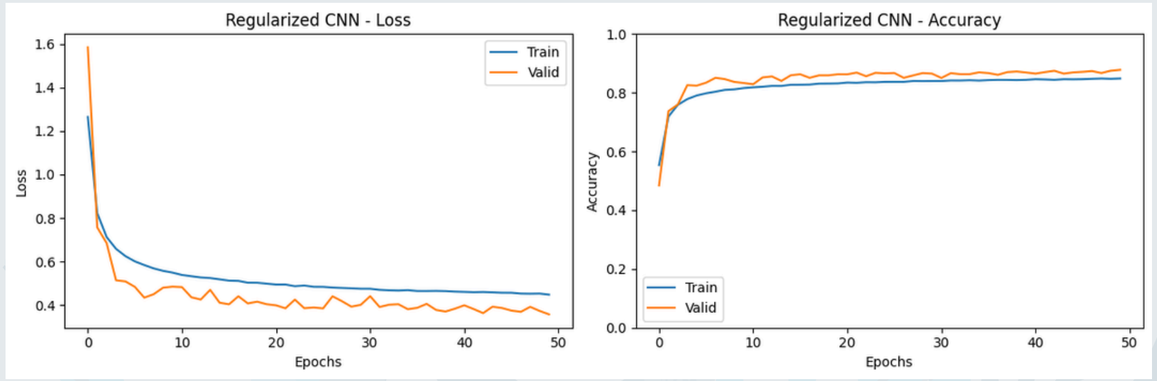
2-block convolutional architecture with 32 and 64 filters, using Global Average Pooling directly before softmax and no dense layers or regularization.



72.5%
Test Accuracy

Regularized CNN

3-block convolutional architecture with 32, 64, and 128 filters, Batch Normalization after each convolution, and Dropout (0.25 per block and 0.5 before the classifier).

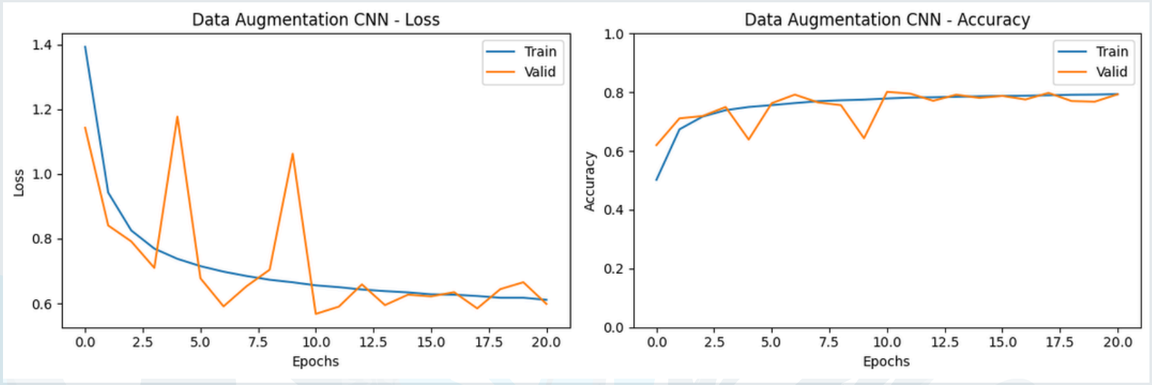


87.9%
Test Accuracy (+15.4 pp)

Data Augmentation & Hyperparameter Tuning

Augmented CNN

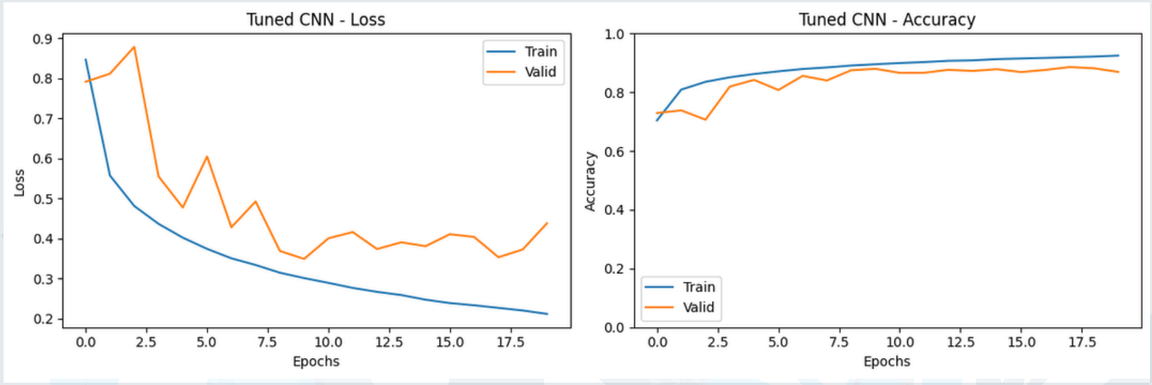
Regularized architecture extended with data-augmentation layers, including horizontal flipping, ±10% translation, ±5° rotation, and ±10% zoom, applied only during training as Keras disables augmentation at inference.



80.6%
Test Accuracy (-8.1% pp)

Tuned CNN (Keras Tuner)

Architecture optimized through hyperparameter search using Keras Tuner, exploring filter counts (32, 64, 96), dropout rates (0.20, 0.25, 0.30), dense-layer sizes (128, 256), and learning rates (1e-3, 5e-4).

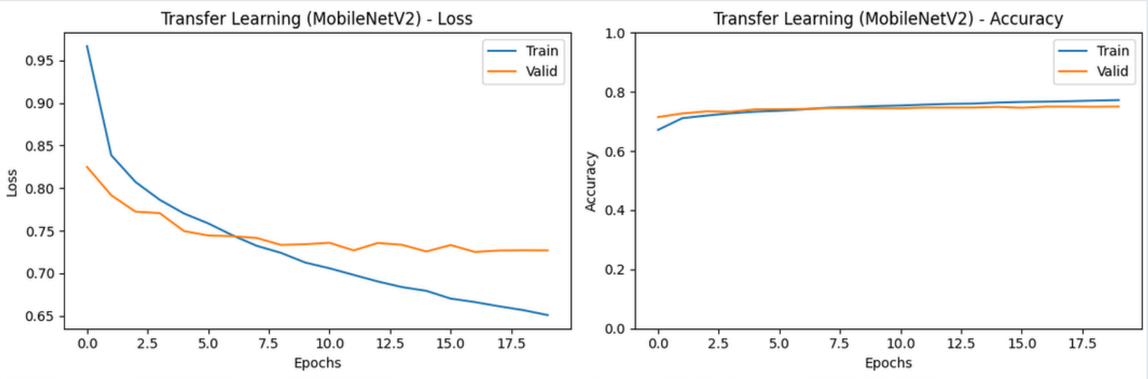


87.9%
Test Accuracy (ties Regularized)

Transfer Learning - MobileNetV2

Frozen Backbone

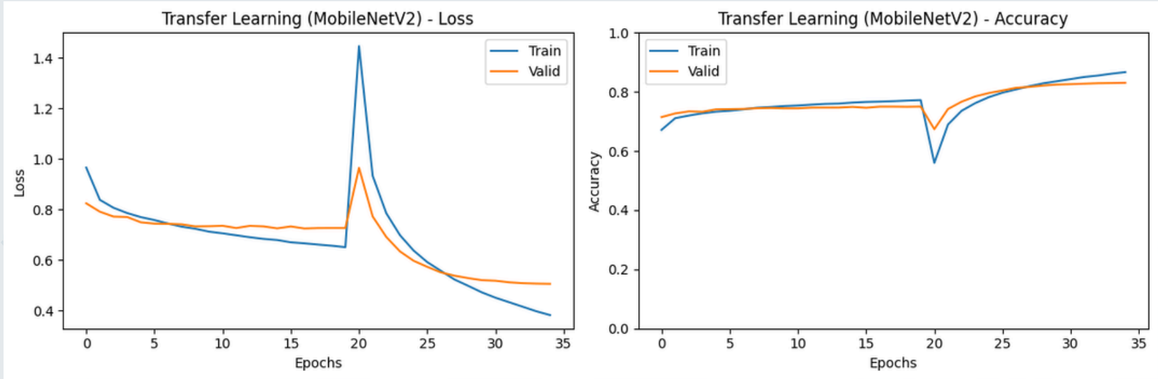
MobileNetV2 pretrained on ImageNet with the classification head removed. The backbone is initially frozen while a custom head is trained.



75.2%
Test Accuracy - Frozen only

Fine-tuned

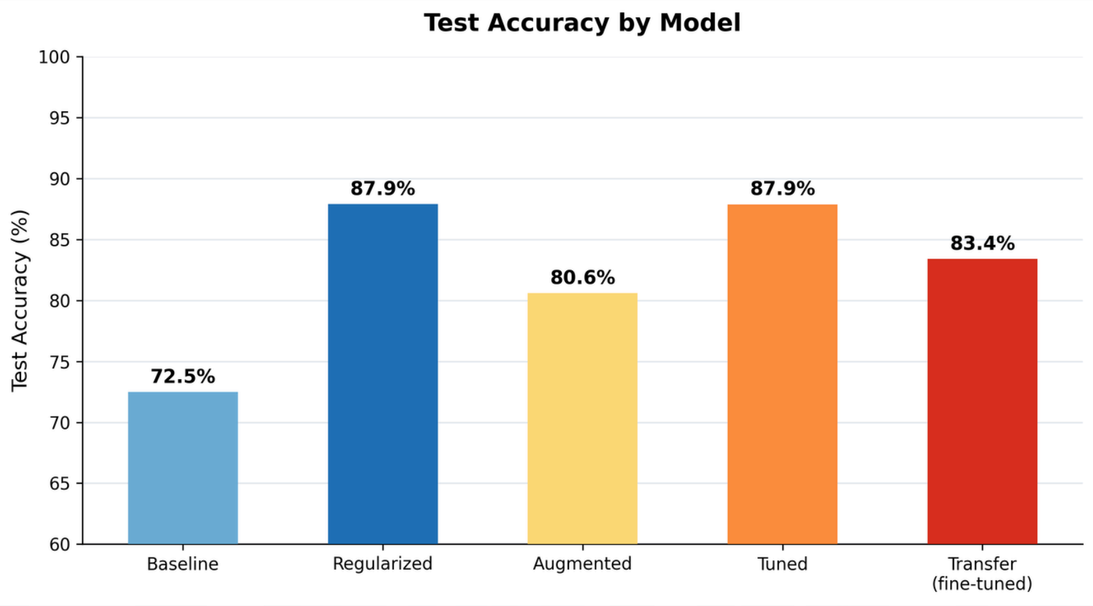
Then, the head is partially unfrozen from block16 onward and fine-tuned with a very low learning rate (1e-5) to preserve pretrained features while adapting upper layers to the sketch domain.



83.4%
Test Accuracy

Results at a Glance

The structured progression showed that neither added capacity nor standard augmentation improved performance on sparse sketches. This led us to adopt a more optimized and parameter-efficient architecture.



Best model: the best-performing models were the Regularized & Tuned CNN, both reaching **87.9% accuracy**.

Augmentation: augmentation reduced accuracy, suggesting that the chosen synthetic transformations did not match the natural variability of human sketches.

Transfer: transfer learning provided limited gains because of the domain gap between natural images and line drawings.

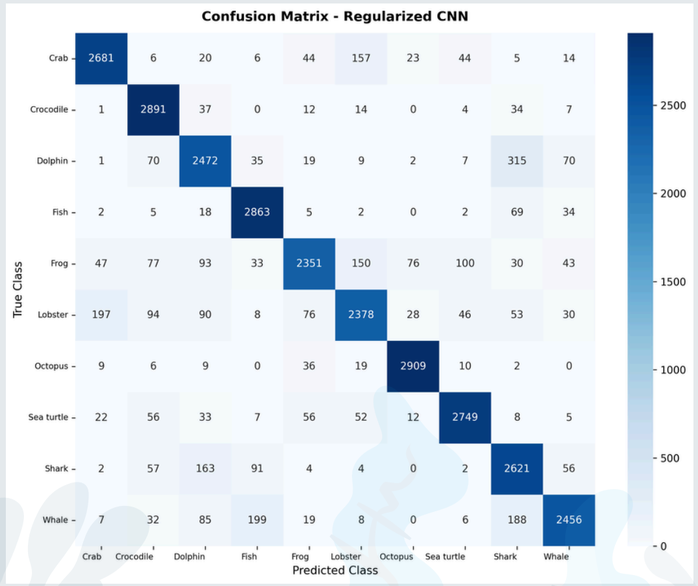
Results in Detail

Quantitative Comparison

Metric	Regularized	Tuned
Test Accuracy	87.90%	87.89%
Test Loss	0.3553	**0.3514**
Best Val Loss	0.3574	**0.3488**
Best Val Acc	87.81%	**88.58%**
# Parameters	**111,178**	933,962

Both models tie on accuracy. While the Tuned CNN slightly outperforms on loss metrics, the Regularized CNN is preferred because it achieves the same performance with lower complexity.

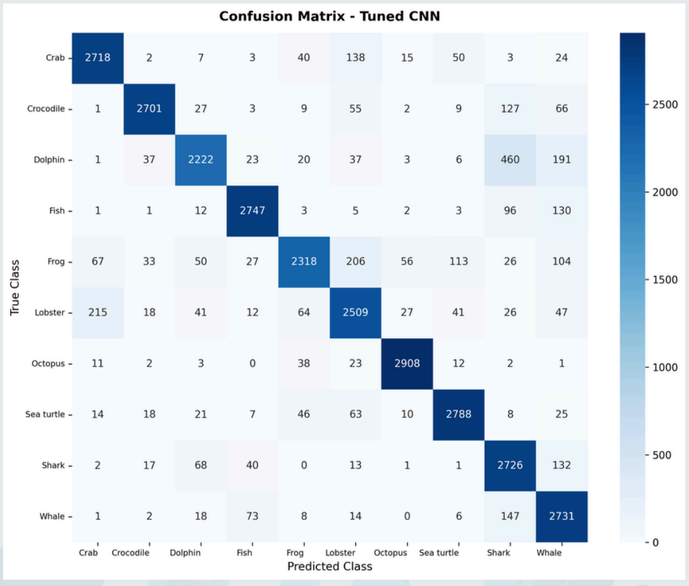
Regularized CNN



Hardest Classes:

Frog: recall 78%, confused with Crab.
Lobster: recall 79%, confused with Crab/Frog.
Dolphin: recall 82%, confused with Shark/Whale.

Tuned CNN



Hardest Classes:

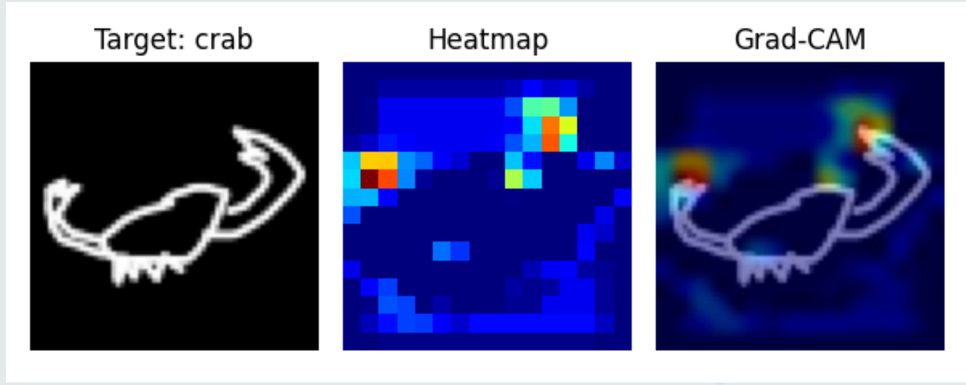
Dolphin: recall 74%, worst class overall.
Frog: recall 77%, similar to Regularized.
Shark: precision 75%, many false positives.

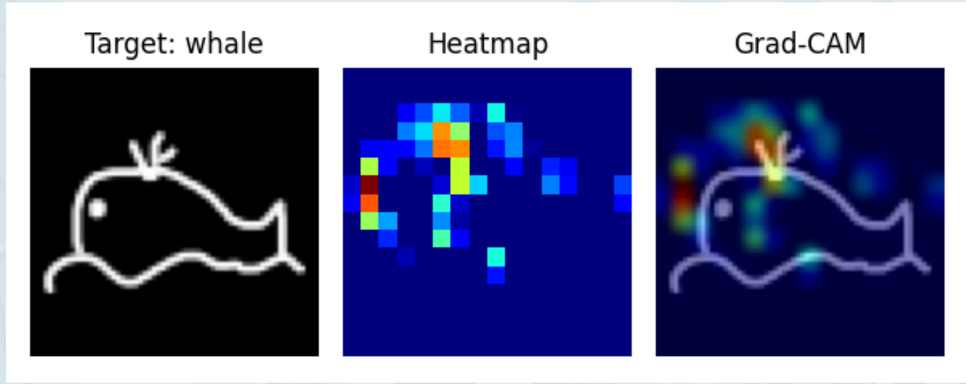
Explainability

Grad-CAM (Gradient-weighted Class Activation Mapping)

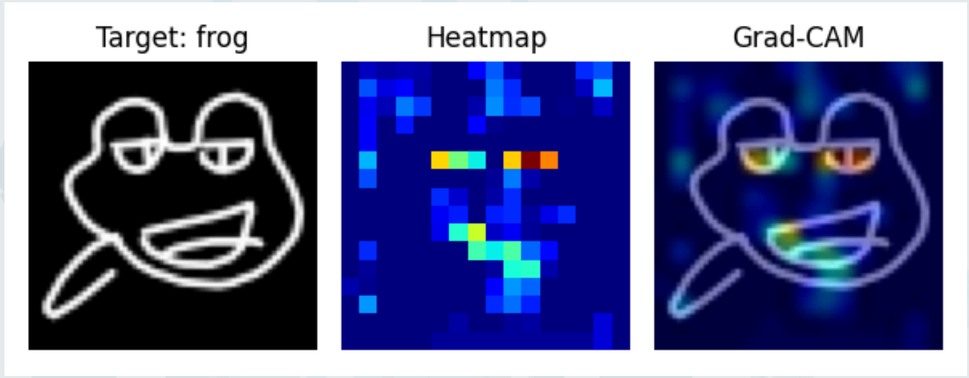
It is an Explainable AI visualization tool that analyses the gradients flowing into the final convolutional layer. In this way, it highlights the specific regions of an image that influence the model's prediction, generating a heatmap that shows where the network focuses its attention.

Regularized



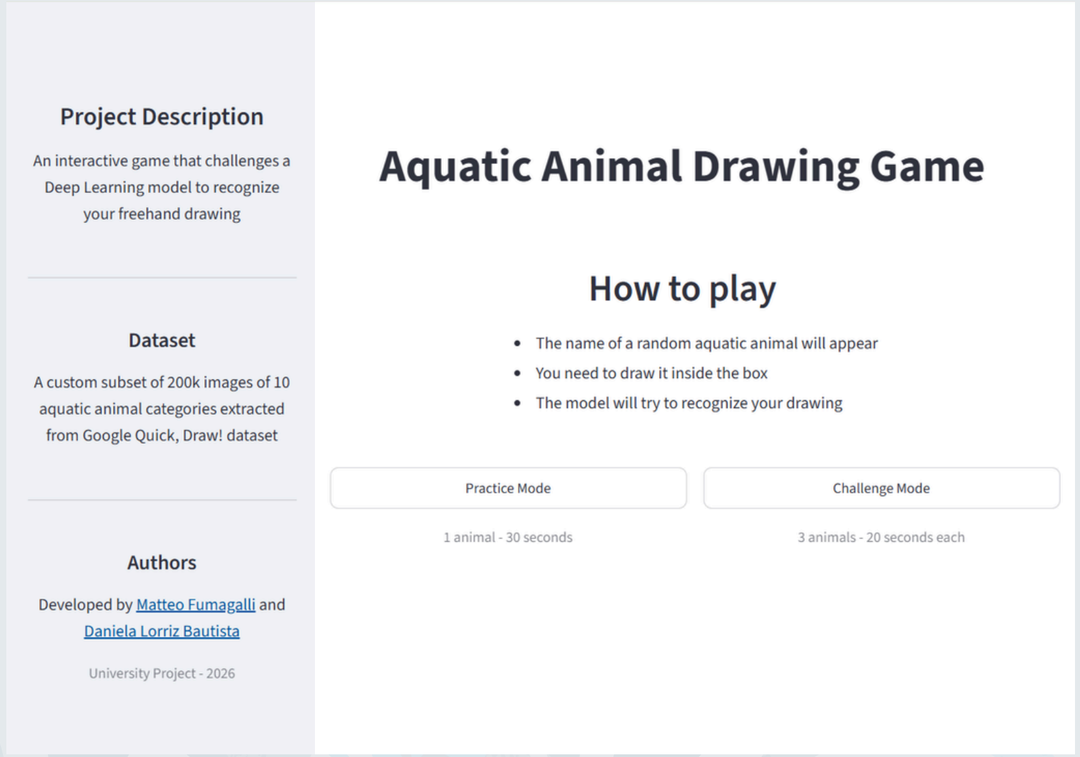


Tuned

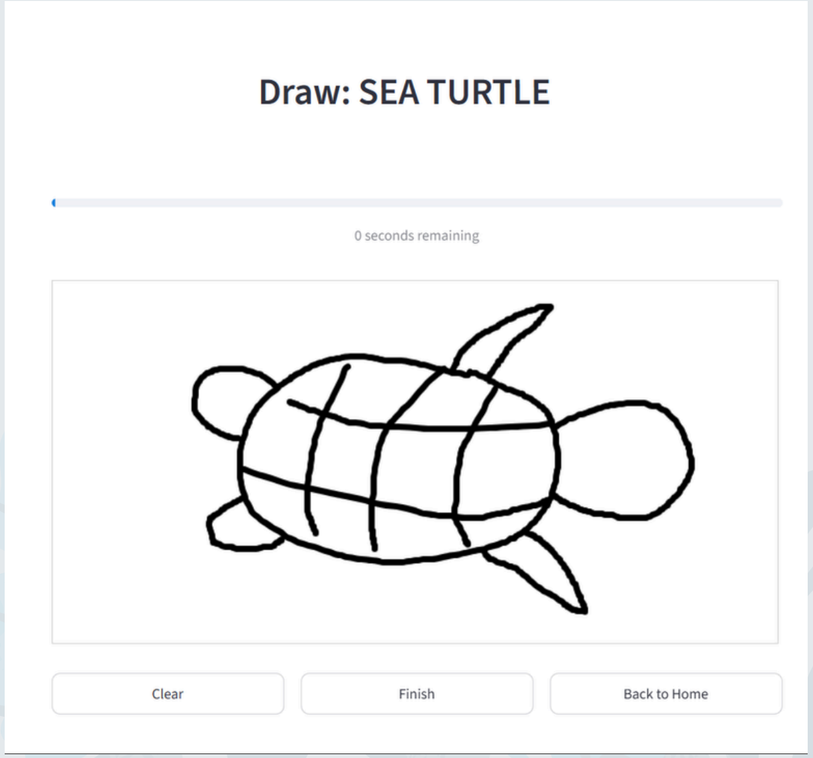


Interactive Model Demo

***Real-time Streamlit* application** that allows users to interact with the model by drawing the indicated aquatic animal and analysing the results of the prediction.



1. Introduction page with mode selection

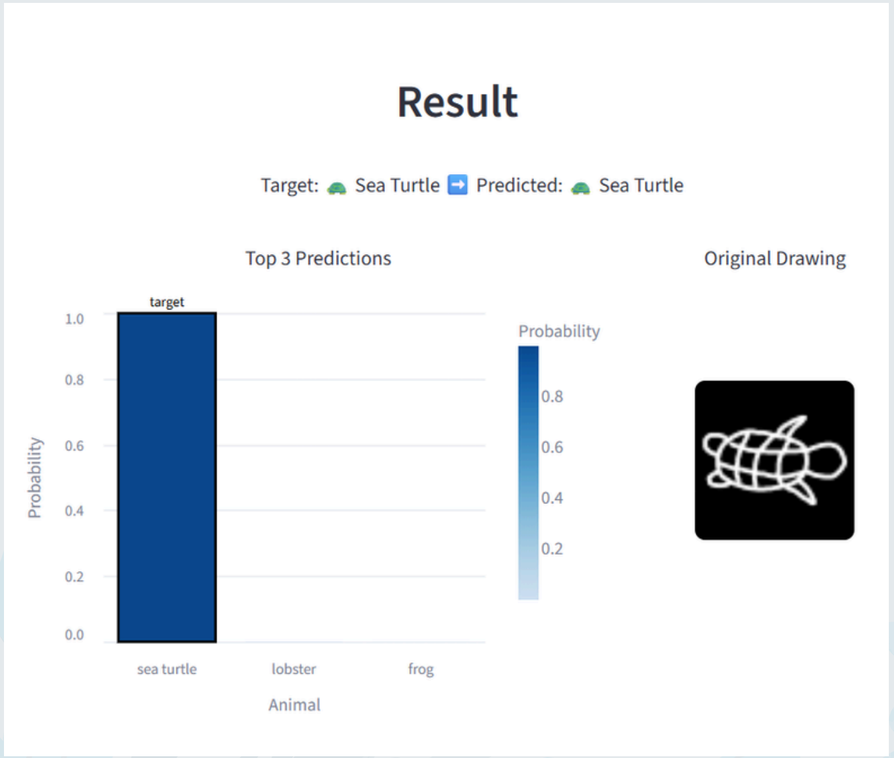


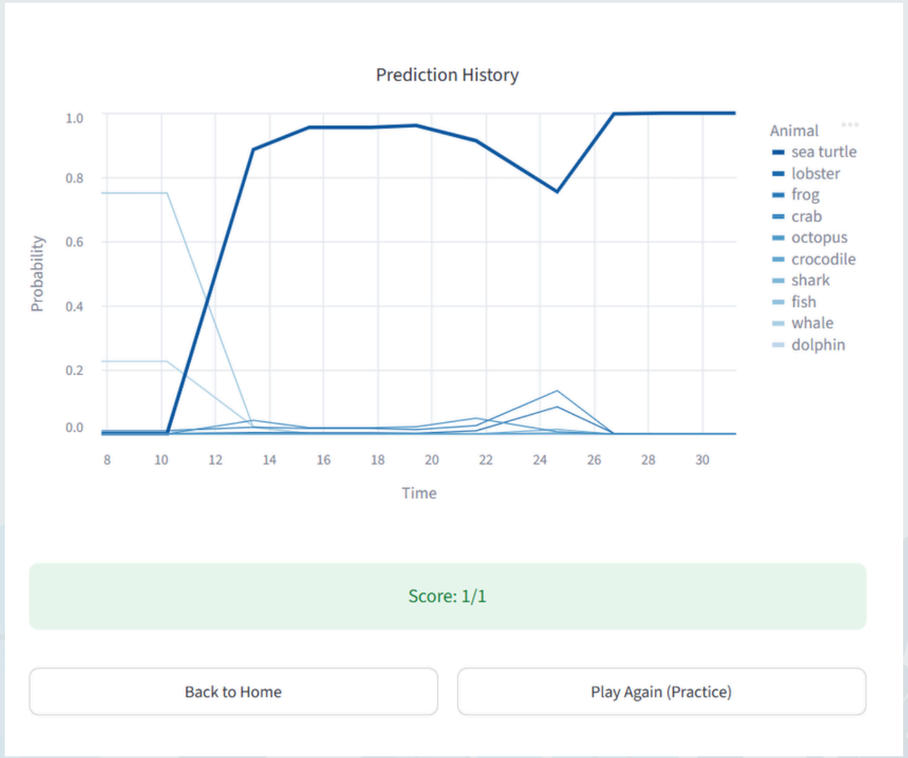
2. Game page with user drawing

PREPROCESSING

The ***st_canvas component*** provides a JSON object based on Fabric.js, defining a path composed of multiple commands:

- M moves the cursor to the point (x, y) without drawing.
- Q draws a freehand curve defined by control and end points.
- L draws a straight segment toward the point (x, y).





3. Prediction page with result

Thank **You**

The deep ocean remains a mysterious, vast, and fascinating world filled with untold secrets waiting to be explored.